RunBaza Л4 Руководство к защите

Hamuev Oleg | CR-232 | Leah A.

Цель защиты: показать, что формы меняют общие данные приложения, а несколько экранов получают обновления через одну модель. Затем объяснить, почему интерфейс не зависит от того, где лежат данные.

Что сказать в начале

«В Л2 я сделал интерфейс, в Л3 добавил навигацию и проверку форм. В Л4 формы начали сохранять данные в репозитории. Общим состоянием управляют ChangeNotifier и Provider. План, счётчики и профиль обновляются после успешной операции. Отдельно я переработал профиль и обработку ошибок».

Что действительно работает

Можно добавить или изменить участие, убрать старт из плана, создать предложение поездки или запрос места, отредактировать профиль и сменить тему. При ошибке запроса есть повторная попытка; форма не теряет заполненные поля.

Что важно не перепутать

Сейчас источник mock: данные живут в памяти приложения. Переключение вкладок не сбрасывает изменения, полный перезапуск сбрасывает. Вход и регистрация проверяют поля локально. Это ещё не аккаунты на сервере и не настоящее бронирование поездок.

Как пользоваться этим гайдом

Страница 2: запуск. Страницы 3-5: показ приложения примерно на 8 минут. Страница 6: схема и код. Страницы 7-8: вопросы преподавателя. Страница 9: Git, проверки и устранение проблем.

Что подготовить заранее

Открой приложение, PDF отчёта и папку проекта в редакторе. Перед показом перезапусти приложение для исходных шести записей плана. Оставь этот гайд рядом. Нужен iOS Simulator, отдельный сервер запускать не требуется.

1 Запуск на Mac и в Xcode

Самый простой запуск через Terminal

```
cd ~/Desktop/PAM/Lab2/run_baza
/Users/oleg.rune/development/flutter/bin/flutter pub get --enforce-lockfile
open -a Simulator
/Users/oleg.rune/development/flutter/bin/flutter devices
/Users/oleg.rune/development/flutter/bin/flutter run -d <simulator-id>
```

Вместо <simulator-id> подставь ID iPhone из flutter devices, без угловых скобок. Команда flutter run должна остаться работающей. В её терминале r делает hot reload, R полный hot restart, q завершает запуск.

Запуск через Xcode

```
open ~/Desktop/PAM/Lab2/run_baza/ios/Runner.xcworkspace
```

Выбери схему Runner и iPhone Simulator вверху окна. Нажми треугольник Run или Cmd+R. Для симулятора подпись устройства не требуется. Если Xcode сообщает об отсутствии Flutter-файлов, сначала выполни flutter pub get и один запуск через Flutter.

Где искать код

```
~/Desktop/PAM/Lab2/run_baza/lib/screens/settings_screen.dart
~/Desktop/PAM/Lab2/run_baza/lib/screens/profile_form_screen.dart
~/Desktop/PAM/Lab2/run_baza/lib/models/collection_model.dart
~/Desktop/PAM/Lab2/run_baza/lib/data/mock_repositories.dart
~/Desktop/PAM/Lab2/run_baza/lib/main.dart
```

Runner в Xcode содержит нативную оболочку iOS. Интерфейс написан на Dart в lib. Чтобы открыть файл, используй редактор или команду open -a Xcode с путём файла. Просто вставлять путь .dart в Terminal нельзя: оболочка попробует запустить его как программу.

Вход перед демонстрацией

Введи oleg@example.com и пароль runbaza26. Это пример корректного ввода, не настоящая серверная учётная запись. Нажми «Войти». Для короткого показа loading открой «Забеги» и нажми кнопку обновления справа вверху.

2 Показ плана и общих счётчиков

Шаг 1 Показать три связанные части

Открой «Мой план»: в начале должно быть 6 стартов. Покажи badge 6 на нижней вкладке. Открой «Профиль», прокрути до «Мой сезон» и покажи те же 6 стартов из цели 8.

Сказать: «Здесь нет трёх независимых копий списка. План хранится в PlanModel. Эти экраны подписаны на одну модель».

Шаг 2 Убрать одно участие

В «Забеги» найди Cricova Wine Cellar Run. Открой детали. Рядом с надписью «В твоём плане» нажми значок удаления закладки. Сначала можно выбрать «Оставить», чтобы показать отмену. Повтори и выбери «Убрать».

Сказать: «Диалог теперь вызывает настоящий delete в mock-репозитории. Удаляется моя запись участия, не событие из общего календаря».

Дождись окончания запроса. Badge станет 5, маркер участия исчезнет. Открой «Мой план» и «Профиль»: там тоже 5. В плане сумма выбранных дистанций станет 86,3 км.

Шаг 3 Добавить участие через форму

Вернись к деталям Cricova. Нажми пустую закладку вверху: она открывает параметры личного плана, даже если продажа слотов ещё не началась. Выбери 10 км, темп 5:30 и «Участникам забега». Нажми «Подтвердить».

Сказать: «Форма передаёт данные модели. Репозиторий присваивает ID, модель добавляет запись и уведомляет подписчиков. Поэтому после закрытия формы счётчик уже обновлён».

Badge снова 6. В деталях видно 10 км. В плане сумма станет 96,3 км. Если ещё раз изменить этот старт, число записей не увеличится: saveForRace обновит существующую запись.

Что означают цифры

Это запланированные старты и дистанции. Они не доказывают, что пользователь пробежал эти километры. Настройка видимости пока хранит выбор пользователя, но не публикует его другим аккаунтам.

3 Показ профиля и формы поездки

Шаг 4 Изменить профиль

Открой «Профиль», нажми карандаш или «Настроить мой профиль». Выбери цвет аватара. Укажи Hamuev Oleg, oleg@example.com, Chișinău. Добавь короткую биографию, темп 5:30, любимую дистанцию 10 км и цель 10. Нажми «Подтвердить».

Сказать: «Профиль разделён на обзор и форму. Данные сохраняются через ProfileModel и ProfileRepository. Карточка и цель обновляются после успешного ответа».

Покажи новый цвет, темп и цель 6 из 10. Прокрути вниз и выбери «Тёмная», затем «Светлая». Тема меняется во всём приложении. «Система» использует настройку устройства.

Шаг 5 Создать поездку

Открой «Попутки», выбери Iaşi Autumn Run, нажми «Я водитель». Сначала отправь пустую форму: обязательные поля подсветятся. Затем заполни пример ниже.

```
Имя: Oleg Hamuev
Город выезда: Chișinău
Место встречи: Circ
Время выезда: 06:30
Свободных мест: 3
Вклад с человека: 100 MDL
Телефон: +37360000000
```

Нажми «Подтвердить». После ожидания откроется список поездок. Найди карточку Oleg Hamuev с этими значениями. Форму пассажира можно показать аналогично: в ней нет цены, создаётся запрос места.

Сказать: «В Л3 кнопка только проверяла форму. Сейчас OfferModel.add вызывает create в репозитории. Новая карточка появилась из общего состояния. Ручной перезагрузки после pop нет».

Проверка ограничений

Время 24:00 и 0 мест должны быть отклонены. В профиле цель 0 или 53 также отклоняется. Кнопка сохранения блокируется на время запроса, поэтому двойное нажатие не должно создавать дубликаты.

4 Управляемая ошибка и восстановление

Шаг 6 Ошибка чтения

Открой «Профиль», прокрути вниз, раскрой «Диагностика». Включи «Симулировать сбой сети». Открой «Забеги» и нажми обновление. Сначала появится loading, затем сообщение с кнопкой «Повторить».

Сказать: «Это управляемая ошибка mock-источника. Приложение обрабатывает Err и предлагает повторить запрос. Я могу воспроизвести сбой независимо от настоящего интернета».

Нажми «Повторить», не выключая сбой: ошибка должна повториться. Вернись в профиль. Если там уже экран ошибки, переключатель всё равно доступен внизу. Выключи его. Вернись к ошибочному списку и нажми «Повторить». Данные вернутся.

Шаг 7 Ошибка сохранения без потери полей

Сначала выключи сбой и восстанови список. Открой форму новой поездки и заполни её. Переключись вкладкой в профиль, включи сбой, затем вернись вкладкой «Попутки». Форма сохранится в своём стеке. Нажми «Подтвердить».

Сказать: «Запрос завершился ошибкой. Форма не закрылась, запись не добавилась, значения контроллеров остались. Пользователь может повторить действие».

Снова выключи сбой в профиле. Вернись к форме и повтори submit. Теперь новая карточка должна появиться ровно один раз. В конце оставь аварийный переключатель выключенным.

Почему для чтения и записи разный интерфейс

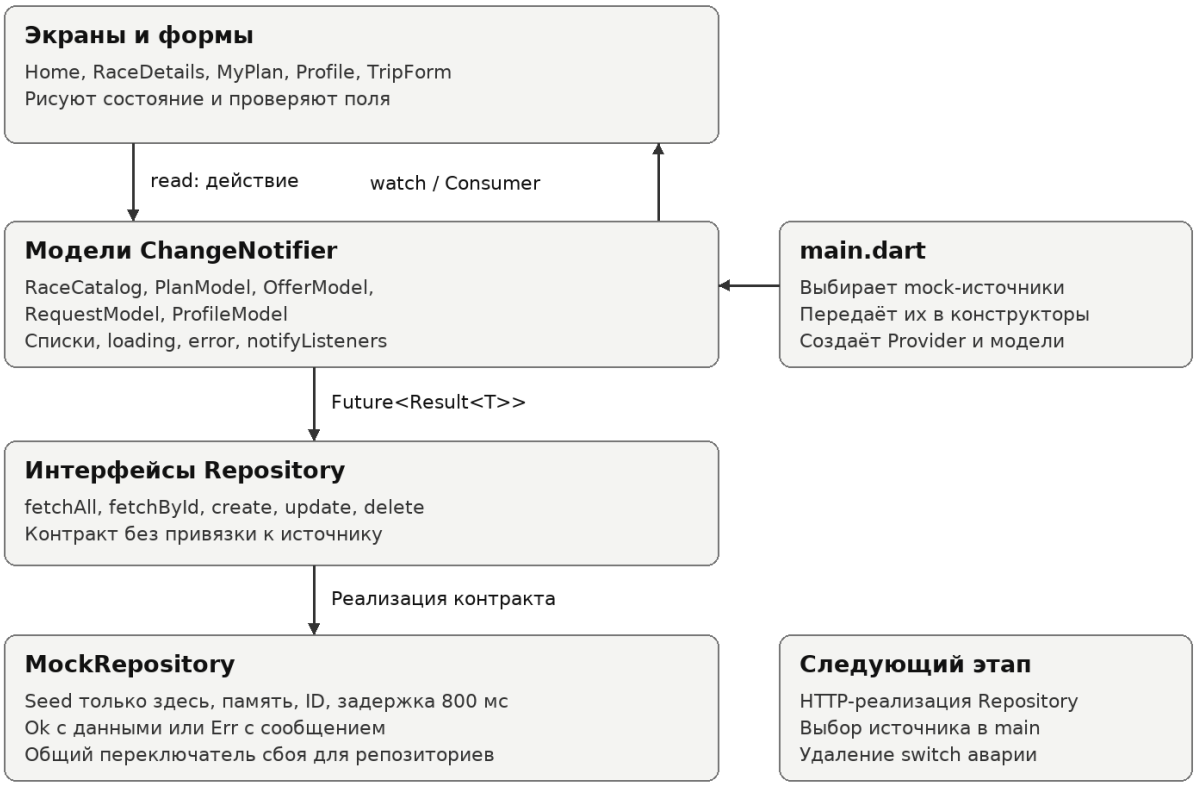
При неудачной загрузке экран показывает ErrorView вместо списка. При неудачной записи оставляет форму и показывает SnackBar. Так пользователь видит причину и не теряет работу. Оба пути получают сообщение из Err.

Если преподаватель просит перезапуск

После полного перезапуска начальные записи вернутся. Объясни, что это договорённость Л4: mock в памяти. Внешнее хранилище появится в следующем этапе. Hot reload обычно сохраняет State и не равен полному перезапуску.

5 Архитектура простыми словами

«Экран отвечает за внешний вид. Модель хранит текущее состояние и сообщает о его изменении. Репозиторий отвечает за чтение и запись. Интерфейс репозитория позволяет заменить память на сервер, не меняя экран».



Локально в State: поля формы, поиск, фильтры и переключатель аварии.

Экраны и модели не импортируют mock_data.dart.

Какие файлы открыть на защите

1. lib/data/repository.dart и result.dart: пять CRUD-методов, Future, Ok и Err.
2. lib/data/mock_repositories.dart: _fails, fetchAll и create; задержка, флаг, ID.
3. lib/models/collection_model.dart: load, records и _write.
4. lib/models/plan_model.dart: add или update по raceId.
5. lib/main.dart: RunBazaScope, Provider и создание моделей.
6. lib/widgets/app_shell.dart: Consumer со счётчиком.
7. lib/screens/participation_form_screen.dart: _save и mounted.

Одна операция по цепочке

Пользователь нажал «Подтвердить». Form проверил поля. read получил модель, модель вызвала create. Mock подождал 800 мс и вернул Ok с новым ID. Модель обновила приватный список и вызвала notifyListeners. Подписанные виджеты перестроились.

6 Вопросы про состояние

Чем watch отличается от read

watch в build подписывает виджет на изменения модели. read в callback получает модель для действия без подписки. Если взять read для badge, счётчик не будет обновляться именно от модели. watch в обычном обработчике события может вызвать проверку неправильного места подписки.

Зачем Consumer

Он подписывает конкретную часть дерева. В RunBaza это badge плана. Остальной NavigationBar не нужно переписывать ради получения количества записей; child может быть заранее построенным значком.

Что будет без notifyListeners

Данные в модели изменятся, но подписчики не получат сигнал перестроиться. Иногда значение появится после другого обновления UI, поэтому баг кажется случайным. В коде уведомления есть до и после загрузки, а также при начале и завершении записи.

Почему нельзя просто setState

setState подходит для локального состояния виджета: поиска, фильтра, скрытия пароля. План нужен нескольким вкладкам. Если каждая хранит свой список, значения разойдутся. Поэтому общий список вынесен в ChangeNotifier.

Почему List.unmodifiable

Экран не может выполнить records.add и изменить список без репозитория и уведомления. Для изменения он должен вызвать метод модели. Доменные сущности также имеют final-поля и copyWith.

Что такое Provider и DI

Provider делает объект доступным потомкам дерева. DI означает, что модель получает Repository через конструктор, а не создаёт конкретный MockRepository внутри. Это упрощает замену источника и тестирование.

Зачем ValueKey с ID

После удаления или вставки Flutter должен понимать, какой записи принадлежит строка. ID стабилен, позиция в списке меняется. UniqueKey каждый раз создавал бы другую идентичность.

7 Вопросы про данные и ошибки

Что такое Future<Result<T>>

Future означает, что результат придёт позже. Result описывает ожидаемый исход: Ok содержит значение типа T, Err содержит сообщение. Future нужен для ожидания, Result для явного успеха или ошибки.

Зачем sealed и switch

Все прямые варианты Result известны в библиотеке. Dart может проверить полноту switch. Здесь обрабатываются Ok и Err. Добавление нового варианта потребует учесть его в местах разбора результата.

Разве исключений больше не бывает

Result делает ожидаемые ошибки частью контракта, но не отменяет ошибок программирования. Будущая HTTP-реализация должна перехватывать ожидаемые транспортные ошибки и возвращать Err. Нельзя считать, что sealed сам ловит исключения.

Зачем задержка 800 мс

Она воспроизводит асинхронное ожидание и позволяет проверить spinner, блокировку submit и поздние ответы. Без задержки переход loading мог бы быть слишком быстрым для демонстрации.

Почему ID назначает репозиторий

Хранилище отвечает за идентичность записей. Форма создаёт черновик, а create возвращает объект с новым ID. В HTTP-версии эту роль может выполнять сервер.

Зачем mounted и номер запроса

mounted проверяется после await перед действиями с экраном. Пользователь мог уйти со страницы. Номер запроса в модели не позволяет старому медленному load перезаписать результат более нового.

Какие файлы менять для backend

Добавить HTTP-реализации Repository, выбрать их в main.dart и убрать mock-переключатель в профиле. При сохранении того же контракта модели и экраны менять не потребуется. Реальная авторизация будет отдельной задачей.

Почему после удаления событие осталось в календаре

Мы удалили PlanEntry, то есть участие пользователя. Race описывает событие организатора. Это разные сущности и разные репозитории.

8 Git проверки и быстрые исправления

История работы

```
cd ~/Desktop/PAM
git status
git log --oneline --decorate -30
git log --oneline main -- Lab2/run_baza
git diff main...lab/4-state-repository --stat
```

Изменения разбиты по реальным этапам: зависимость provider, контракты и mock, общие модели, подключение экранов, формы, профиль и тесты. Итоговые документы добавлены отдельным коммитом. После слияния diff с main может быть пустым, это нормально.

Что проверено

125 unit/widget-тестов, анализ без замечаний, iOS Simulator build и сценарий Л4 на iPhone 17 Pro. GitHub Actions отдельно проверяет оба Flutter-проекта. Итог CI виден во вкладке Checks у pull request.

```
cd ~/Desktop/PAM/Lab2/run_baza
flutter analyze --fatal-infos
flutter test
rg -n "mock_data" lib/screens lib/widgets lib/models
```

Последняя команда должна не найти совпадений. Код UI получает данные через модели, seed используется только mock-репозиториями. Это одна из отдельных проверок задания.

Если что-то пошло не так

Нет команды flutter: используй полный путь /Users/oleg.rune/development/flutter/bin/flutter.
Не видно iPhone: открой Simulator и снова выполни flutter devices.
Постоянно ошибка данных: выключи «Симулировать сбой сети», затем нажми «Повторить».
Не видно нижней кнопки: прокрути форму, закрой клавиатуру.
После входа другое имя: login задаёт локальный e-mail сеанса; имя можно сохранить в профиле.
После перезапуска пропали изменения: это ожидаемое поведение mock в Л4.

Финальная фраза

«Главный результат Л4: единый источник состояния для нескольких экранов, асинхронные репозитории и явные состояния ожидания и ошибки. Изменения проходят через модель, а UI обновляется по уведомлению».

Справка: pub.dev/packages/provider; api.flutter.dev/flutter/foundation/ChangeNotifier-class.html; dart.dev/language/class-modifiers#sealed. Сценарий и пути сверены с кодом RunBaza Л4.